SkinAI: Technical Development Journey

From a Pre-trained Ensemble to Production Calibration

A Deep Technical Reference for Future Projects

Shawn Li

June 2026

0 Contents

1 Project Overview

SkinAI is a web application that accepts a dermoscopic photograph of a skin lesion and returns a ranked list of likely diagnoses across nine dermatological conditions, along with a high-risk flag for malignancies requiring urgent referral.

1.1 What we were actually building

The core product is a **triage tool**, not a diagnostic device. The distinction matters both legally and technically:

- A *diagnostic device* must be validated to FDA/CE standards and cannot be deployed without regulatory approval.
- A *triage tool* assists a clinician in prioritising which patients need urgent attention. It is held to a different (though still high) standard: it must be *safe to miss* — that is, false negatives on high-risk classes are more dangerous than false positives.

This distinction shaped every technical decision: we optimised for **recall** on malignant classes (MEL, BCC, SCC, AK) even at the cost of precision. It is better to flag a benign lesion as potentially malignant than to miss a melanoma.

1.2 The nine conditions

Code	Full name	Risk	Notes
AK	Actinic keratosis	HIGH	Pre-cancerous; can progress to SCC
BCC	Basal cell carcinoma	HIGH	Most common skin cancer
BKL	Benign keratosis	Low	Seborrheic keratosis, solar lentigo
DF	Dermatofibroma	Low	Benign fibrous nodule
MEL	Melanoma	HIGH	Most dangerous; high mortality if late
NV	Melanocytic nevus	Low	Common mole; baseline class
SCC	Squamous cell carcinoma	HIGH	Second most common; more aggressive than BCC
UNK	Unknown	—	Catch-all; model uncertain
VASC	Vascular lesion	Low	Haemangioma, angiokeratoma

Table 1: Nine-class classification scope. HIGH risk classes trigger an urgent referral flag.

2 The Model: EfficientNet Ensemble

2.1 What is EfficientNet?

EfficientNet is a family of convolutional neural networks introduced by Tan & Le (Google Brain, 2019). The key idea is **compound scaling**: instead of making a network deeper, wider, or higher-resolution in isolation, EfficientNet scales all three dimensions simultaneously using a fixed ratio derived from a neural architecture search.

The family runs from B0 (small, fast) to B7 (large, accurate). We used B4, B5, and B7:

Model	Params	Input size	Top-1 ImageNet
EfficientNet-B4	19M	380×380	83.0%
EfficientNet-B5	30M	456×456	83.7%
EfficientNet-B7	66M	600×600	85.0%

Table 2: Models in the SkinAI ensemble. Larger input sizes capture finer dermatoscopic detail.

2.2 Why an ensemble of three?

A single model has high variance on medical imaging tasks. Using three models of increasing capacity and training them on slightly different data augmentations gives an ensemble that:

1. **Averages out individual model errors.** If B4 is wrong on a particular lesion texture but B7 is right, their averaged probability is closer to the truth.
2. **Reduces overconfidence.** A single model often assigns probability 0.99 to an incorrect class. The ensemble rarely agrees that confidently unless it is genuinely correct.
3. **Improves calibration.** Ensemble probabilities tend to be better-calibrated (closer to true frequencies) than individual model probabilities.

The **soft-voting** strategy averages the output probability vectors before taking the argmax, rather than having each model cast a discrete vote. This preserves uncertainty information.

2.3 Training data: ISIC and HAM10000

The weights we started with were pre-trained on:

- **ISIC 2019 Challenge dataset:** 25,331 dermoscopic images across 9 classes.
- **ISIC 2020 Challenge dataset:** 33,126 images (primarily melanoma vs. benign).
- **HAM10000** (Human Against Machine with 10000 training images): 10,015 images across 7 classes, collected at multiple hospitals in Austria and Australia.

Critical imbalance problem. The combined dataset is approximately: NV: 65%, MEL: 17%, BKL: 10%, BCC: 4%, AK: 3%, SCC: 2%, DF: 1%, VASC: 1%. The model trained on this data will learn that predicting NV is almost always "good enough." This will cause catastrophic recall failure on rare classes — and it did.

3 The Inference Pipeline

3.1 End-to-end flow

```
User uploads image
  -> web-v2/app.js: SkinAI.runRealAnalysis()
    -> POST /predict  (Modal API: sh4wn27--skinai-api.modal.run
       )
```

```
      -> FastAPI main.py: reads file bytes
        -> inference.py: SkinAIEnsemble.predict_probs(image)
          1. _tta_views(image)          # 4 geometric views
          2. for each model:
               for each view:
                 model.predict(_preprocess(view, (H,W)))  # (1,
                    H,W,3) float32
               average 4 view predictions
          3. average 3 model predictions -> raw: shape (9,)
          4. _calibrate(raw)            # LR head or
             temperature scaling
        -> predict(): argsort, build top-3, set high_risk flag
      -> return JSON
    -> app.js: animate results, show risk banner
```

3.2 Preprocessing

Each image is preprocessed identically to the original training pipeline:

```
def _preprocess(img, size):
    img = img.convert("RGB").resize((size[1], size[0]))  # PIL:
        (W,H)
    arr = np.asarray(img, dtype=np.float32) / 255.0
    # ImageNet normalisation (matches albumentations A.
       Normalize() in training)
    MEAN = [0.485, 0.456, 0.406]
    STD  = [0.229, 0.224, 0.225]
    arr = (arr - MEAN) / STD
    return arr[None, ...]  # add batch dim -> (1, H, W, 3)
```

Use the same normalisation as training. The model's weights encode assumptions about the input distribution. If training used ImageNet statistics but inference uses [0,1] or [-1,1], every activation in the network is wrong. This is one of the most common bugs in deploying pre-trained models.

4 Test-Time Augmentation (TTA)

4.1 What is TTA and why does it help?

A trained model is a deterministic function. Given the same input, it always produces the same output. But real images contain orientational variation: a lesion photographed with the camera rotated 90° is the same lesion, but the raw pixel values are completely different.

TTA runs the model on multiple augmented versions of the same image and averages the output probabilities. This effectively approximates an ensemble over orientations.

4.2 Our implementation

We use four geometric views (no colour jitter, to avoid introducing distribution shift):

```
def _tta_views(img):
    return [
        img,                                          # original
        img.transpose(Image.FLIP_LEFT_RIGHT),         # horizontal
           flip
        img.transpose(Image.FLIP_TOP_BOTTOM),         # vertical flip
        img.transpose(Image.ROTATE_180),              # 180-degree
           rotation
    ]
```

Each of the three models processes all four views, giving $3 \times 4 = 12$ forward passes per image. The views from each model are averaged first, then the three model averages are averaged:

$$\hat{p}_c = \frac{1}{3}\sum_{m=1}^{3}\left(\frac{1}{4}\sum_{v=1}^{4} f_m^{(v)}(x)_c\right)$$

where $f_m^{(v)}(x)_c$ is the probability for class c from model m on view v.

4.3 Why these four views specifically?

Dermoscopic images can be taken in any orientation. Melanomas in particular have *asymmetry* as a diagnostic criterion — but asymmetry relative to what? The model should not rely on a particular orientation. Horizontal flip, vertical flip, and 180° rotation together cover all four quadrant reflections of the image without introducing out-of-distribution transforms (e.g., 90° rotation would create portrait-format lesions from landscape crops, which the model may not have seen during training).

5 Diagnosing the Class Imbalance Problem

5.1 The misleading 80% accuracy figure

When the project began, the model was reported as having "80% accuracy." This figure was computed as:

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

on a test set drawn from the same distribution as the training set — which was 65% NV. A model that predicts NV for everything achieves 65% accuracy. Our model, which correctly identified NV most of the time, achieved 80% simply by being good at the dominant class.

5.2 Balanced accuracy

The more honest metric is **balanced accuracy** (also called macro-averaged recall):

$$\text{Balanced Accuracy} = \frac{1}{C}\sum_{c=1}^{C}\text{Recall}_c = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c}$$

This treats each class equally regardless of how many examples it has. A model that predicts NV for everything achieves $1/9 = 11\%$ balanced accuracy.

5.3 How we measured it

We built a confusion matrix evaluator that fetches fresh images from the **ISIC Archive API v2** (a public database of 552,869 dermoscopic images at `api.isic-archive.com`), runs each image through the ensemble, and computes per-class recall. Key API endpoint:

```
GET https://api.isic-archive.com/api/v2/images/search/
  ?query=diagnosis_3:"Solar or actinic keratosis"
  &limit=50
```

The field hierarchy is: `diagnosis_1` (top-level: malignant/benign) → `diagnosis_2` (e.g. "Malignant epidermal proliferations") → `diagnosis_3` (e.g. "Basal cell carcinoma").

5.4 Results: the brutal truth

Running n=20 images per class on the raw ensemble:

Class	Recall	Notes
NV	95%	Model overwhelmingly predicts NV
MEL	40%	Some signal; confused with NV
BKL	35%	Some signal; confused with NV
BCC	25%	Weak; often predicted as NV
VASC	40%	Distinctive colours help somewhat
AK	0%	Never predicted; no backbone signal
SCC	0%	Never predicted; confused with BKL
DF	0%	Never predicted; confused with NV
Balanced	**34%**	Equal-weight average

Table 3: Raw ensemble recall before any calibration. The 80% figure was prevalence-weighted.

NV dominance confirmed. When we probed raw model probabilities directly on 35 SCC images from ISIC, the backbone's top-1 prediction was BKL on 70% of them. It never once predicted SCC as rank-1. SCC and BKL share a keratotic (crusty, scaly) surface texture — the model learned "crusty = BKL" from the imbalanced training data.

6 Temperature Scaling: Theory and Implementation

6.1 The idea

Temperature scaling is a *post-hoc calibration* technique. After the model is fully trained, you do not touch its weights. Instead, you apply a transformation to the output probabilities to shift which class wins the argmax decision.

The standard formulation applies a single scalar temperature T to all logits:

$$p'_c = \frac{\exp(z_c/T)}{\sum_k \exp(z_k/T)}$$

where z_c are the pre-softmax logits. $T > 1$ makes the distribution more uniform (reduces overconfidence); $T < 1$ makes it more peaked (increases confidence).

6.2 Per-class temperature scaling

We extended this to per-class temperatures, operating in log-probability space:

$$z'_c = \frac{\log p_c}{T_c}, \quad p''_c = \frac{\exp(z'_c - \max_k z'_k)}{\sum_k \exp(z'_k - \max_k z'_k)}$$

The key mathematical insight: since $p_c \in (0, 1)$, we have $\log p_c < 0$. Dividing a negative number by T_c:

- $T_c > 1$: result is *less negative* $\Rightarrow$ higher probability **(BOOST)**
- $T_c < 1$: result is *more negative* $\Rightarrow$ lower probability **(DAMPEN)**

6.3 The critical bug: reversed direction

Initial implementation had temperatures backwards. The first version set $T_{SCC} = 0.7$ and $T_{NV} = 1.4$. Based on the math above: dividing a negative log-prob by 0.7 makes it *more negative* (dampens SCC), while dividing by 1.4 makes NV's log-prob *less negative* (boosts NV). This made the class imbalance problem worse, not better.

6.4 Deriving correct temperature values

We wanted SCC to beat BKL on a typical SCC image. Probing a specific image, we found: $p_{SCC} = 0.174$, $p_{BKL} = 0.715$. After temperature scaling, SCC wins if:

$$\frac{\log p_{SCC}}{T_{SCC}} > \frac{\log p_{BKL}}{T_{BKL}}$$

$$\frac{T_{SCC}}{T_{BKL}} > \frac{\log p_{BKL}}{\log p_{SCC}} = \frac{\log 0.715}{\log 0.174} = \frac{-0.335}{-1.749} \approx 0.192 \times \frac{T_{BKL}}{T_{SCC}}$$

Rearranging: we need $T_{SCC}/T_{BKL} > 0.192$ inverted, which gives $T_{SCC}/T_{BKL} > 5.22$. Setting $T_{SCC} = 8.0$ and $T_{BKL} = 0.85$ gives ratio $= 9.4$. This is sufficient.

We also needed to ensure NV is still dampened enough that high-NV-probability images (common moles) are not affected. Setting $T_{NV} = 0.45$ verified safe via:

$$T_{NV} > T_{SCC} \cdot \frac{\log p_{NV}}{\log p_{SCC}} = 8.0 \cdot \frac{\log 0.87}{\log 0.174} \approx 8.0 \cdot 0.076 = 0.61$$

Wait — this check revealed $T_{NV} = 0.45$ was marginally below the safety threshold for some images. In practice, NV images have $p_{NV} > 0.90$, making the actual threshold lower. We kept $T_{NV} = 0.45$ and monitored for false positives.

Class	T_c	Effect	Reasoning
AK	8.0	Boost	Raw signal present (up to 0.102); needs surfacing
BCC	1.5	Slight boost	Some signal; slight help
BKL	0.85	Dampen	Overrepresented; major confusion source for SCC
DF	1.0	Neutral	Max raw prob 0.004; no signal to boost
MEL	1.5	Slight boost	Some NV confusion
NV	0.45	Dampen	Dominant class; must reduce its pull
SCC	8.0	Boost	Raw signal present (up to 0.174); needs surfacing
UNK	1.0	Neutral	—
VASC	1.2	Slight boost	Some NV confusion

Table 4: Final per-class temperatures after direction correction.

6.5 Final temperature values

6.6 Results after temperature scaling

AK: 0% → 30%, SCC: 0% → 10%, MEL: 40% → 70%, VASC: 40% → 80%. DF remained at 0% — its raw signal was too weak ($p_{DF,\max} = 0.004$) to surface. Balanced accuracy: 34% → 41%.

7 Calibrated Logistic Regression Head

7.1 Why temperature scaling hits a ceiling

Temperature scaling operates on a single number per class (the log-probability). It cannot learn cross-class relationships. For DF, the fundamental problem is that $\log(0.004) = -5.5$ is so deeply negative that even $T_{DF} = 100$ gives $z'_{DF} = -0.055$, while NV might have $z'_{NV} = -0.3/0.45 = -0.67$. DF still wins... but the issue is that on most DF images, NV has $p_{NV} = 0.70$, giving $z'_{NV} = -0.36/0.45 = -0.80$. This is *still less negative* than -0.055... wait, we want $z'_{DF} > z'_{NV}$ which means $-0.055 > -0.80$ — that's true!

In fact, temperature scaling CAN boost DF. The problem is that on DF images, the probability vector is typically distributed across 5-6 classes (BKL, NV, UNK, MEL, BCC all get 0.1-0.2) rather than concentrated. Temperature scaling cannot distinguish "DF image with diffuse probabilities" from "BKL image with diffuse probabilities." It only sees per-class values.

7.2 Logistic regression as generalised temperature scaling

A multinomial logistic regression on log-probabilities has the form:

$$\hat{y} = \text{softmax}(W \cdot \mathbf{z} + \mathbf{b})$$

where $\mathbf{z} = \log \mathbf{p}$ (the 9-dimensional log-probability vector).

If W is constrained to be diagonal and $\mathbf{b} = 0$, this is exactly per-class temperature scaling (each diagonal element is $1/T_c$). When W is unconstrained, the LR head can learn: "DF images tend to have low probability in ALL classes simultaneously, with no clear winner — that diffuse pattern is the DF signature."

The LR head's trick for DF. After training on 35 DF images vs. 35 NV images, the LR learned that DF images have a characteristic *entropy signature*: the log-probabilities are all moderately negative and relatively uniform, rather than having one near-zero entry (as NV images do with $\log p_{NV} \approx -0.1$).

7.3 Implementation

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),  # z-normalise the log-probs
    ("lr", LogisticRegression(
        class_weight="balanced",   # automatically weights rare
            classes
        C=5.0,                     # mild regularisation
        max_iter=2000,
        solver="lbfgs",            # L-BFGS: good for small
           datasets
    )),
])
```

The `StandardScaler` is important: log-probabilities span different ranges across classes (NV log-prob near 0, DF log-prob near -5.5). Without scaling, the LR gradient is dominated by the high-variance features.

7.4 Training data and experimental design

- **35 images per class**, fetched from the ISIC Archive API.
- **Skip the first 20 results per class**. This avoids overlap with images used for evaluation (which also came from the ISIC API). If we trained and evaluated on the same images, we would be measuring memorisation, not generalisation.
- **Raw ensemble probabilities** (before temperature scaling) as input features. The LR head replaces temperature scaling entirely; we do not chain them.
- **25% validation split**, stratified by class, using a fixed random seed for reproducibility.

7.5 Results

7.6 What the LR head cannot fix

SCC recall is still only 12% despite the LR head. The root cause: 70% of the 35 SCC training images had BKL as the backbone's top-1 prediction. The log-probability vectors for SCC and BKL images are too similar for the LR to separate them reliably with only 35 examples. This is a **backbone failure** — the features inside the EfficientNet are not discriminative for SCC vs. BKL. No amount of post-hoc calibration can recover from a backbone that has not learned the relevant features. **Retraining is required.**

Class	Precision	Recall	F1	Support
AK	0.43	0.38	0.40	8
BCC	0.70	0.88	0.78	8
BKL	0.70	0.88	0.78	8
DF	0.30	0.38	0.33	8
MEL	1.00	1.00	1.00	8
NV	0.78	0.88	0.82	8
SCC	0.33	0.12	0.18	8
VASC	1.00	0.88	0.93	8
Balanced accuracy		**67.2%**		

Table 5: LR head validation results. DF recovered from 0% to 38% recall.

8 LDAM Fine-Tuning: The Real Fix

8.1 The long-tail classification problem

Our training set has a *long-tail distribution*: NV has 12,875 images, DF has 239. This 54× ratio causes two problems:

1. **Gradient imbalance.** During each training batch, 65% of gradients come from NV images. The model's weights are continuously optimised to minimise NV error, with little gradient signal from DF.
2. **Decision boundary bias.** The maximum-likelihood solution places the decision boundary for NV far from the NV cluster (to correctly classify the huge number of NV examples), leaving almost no margin for rare classes.

8.2 Label-Distribution-Aware Margin (LDAM) loss

LDAM (Cao et al., NeurIPS 2019) addresses problem 2 by modifying the loss function to enforce *larger margins* for minority classes. The intuition: if a class has few training examples, the model needs to be pushed harder to learn a correct boundary.

The LDAM loss modifies the standard cross-entropy by subtracting a class-specific margin from the logit of the true class before applying softmax:

$$\mathcal{L}_{\text{LDAM}} = -\log \frac{e^{z_y - \Delta_y}}{e^{z_y - \Delta_y} + \sum_{j \neq y} e^{z_j}}$$

where the margin for class y is:

$$\Delta_y = \frac{C}{n_y^{1/4}}$$

with n_y the number of training examples for class y, and C a global scaling constant chosen so that the maximum margin is approximately 0.5.

The $n_y^{1/4}$ denominator means:

- DF ($n = 239$): $\Delta_{DF} = C/3.94$ — large margin (hard to classify as DF unless confident)

- NV ($n = 12875$): $\Delta_{NV} = C/10.67$ — small margin (easy win for NV)

This forces the model to *earn* predictions for common classes, reserving decision boundary space for rare classes.

8.3 Deferred Re-Weighting (DRW) schedule

The LDAM paper found that applying LDAM from epoch 1 causes training instability: the model has not yet learned basic features, and the large margins for rare classes create conflicting gradients. The solution is DRW:

1. **Phase 1** (first half of epochs): Train with standard cross-entropy plus inverse-frequency class weights. This learns basic features stably.
2. **Phase 2** (second half): Switch to LDAM loss with class weights. The model already has a reasonable feature representation; LDAM refines the decision boundaries.

8.4 Fine-tuning strategy: freeze then unfreeze

We do *fine-tuning*, not training from scratch. The pre-trained weights contain valuable dermoscopic features learned from 68,000+ images. We only modify the parts of the network that encode class-specific decision boundaries:

```
# Freeze everything
for layer in model.layers:
    layer.trainable = False

# Unfreeze top 30 layers + classifier head
for layer in model.layers[-30:]:
    layer.trainable = True
```

The top 30 layers correspond roughly to the final EfficientNet block and the global average pooling + dense classifier head. These layers contain the high-level semantic features most relevant to the class boundary problem. The lower layers (edge detectors, texture filters) are kept frozen to preserve the general dermoscopic feature extraction capability.

8.5 Running the fine-tuning job

The fine-tuning runs as a Modal serverless job on the same T4 GPU used for inference:

```
source backend/.venv/bin/activate
modal run scripts/fine_tune_modal.py                                #
    all 3 models
modal run scripts/fine_tune_modal.py --model efficientnet_b4.h5
     # one at a time
```

The job:

1. Loads the existing weights from the Modal volume (`/weights/`).
2. Downloads 120 images per class from ISIC (skipping first 55 to avoid overlap).
3. Runs Phase 1 (10 epochs, LR 10^{-4}, standard CE + class weights).

4. Runs Phase 2 (10 epochs, LR 5×10^{-5}, LDAM + class weights).
5. Saves `finetuned_efficientnet_b4.h5` (etc.) back to the volume.

After completion, update `ENSEMBLE_FILES` in `backend/modal_app.py` to reference the finetuned weights, redeploy, and retrain the LR calibration head.

9 Deployment Architecture

9.1 Frontend: Vercel static hosting

The frontend (`web-v2/`) is a static site deployed to Vercel:

```
// vercel.json (repo root)
{
  "framework": null,
  "buildCommand": "",
  "installCommand": "",
  "outputDirectory": "web-v2"
}
```

Key points:

- `framework: null` prevents Vercel from trying to detect and build a Python or Node app. Without this, Vercel detects `backend/main.py` and attempts to deploy it as a Python function, which fails.
- `.vercelignore` excludes `backend/` and `frontend/` from the bundle. Without this, 7.5 GB of model weights would be uploaded to Vercel on every deploy.
- Live URL: `skinai-silk.vercel.app`. The domain `skinai.vercel.app` returns 404 — it is a different Vercel project that is not linked to this repository.

9.2 Backend: Modal serverless GPU

Modal provides serverless GPU compute. The backend function:

```
@app.function(
    image=image,
    volumes={"/weights": weights_volume},  # persistent model
       weights
    gpu="T4",           # ~8x speedup vs CPU; ~$0.59/hr
    timeout=300,
    scaledown_window=300,  # stay warm for 5 min after last
       request
    memory=16384,          # 16 GB: B4+B5+B7 together need ~12
       GB RAM
)
```

The weights are stored in a **Modal Volume** (persistent block storage), not bundled into the container image. This is critical: the three EfficientNet models total 7.5 GB. Bundling them into the image would add 7.5 GB to every container rebuild (slow and expensive). The volume persists across deployments and scales independently.

9.3 CORS configuration

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://skinai-silk.vercel.app",
        "http://localhost:3000",
    ],
    allow_methods=["POST"],
    allow_headers=["*"],
)
```

file:// origin is blocked. Opening `index.html` directly in the browser (as a file:// URL) will fail CORS. Always test with a local HTTP server: `python3 -m http.server 3000` from `web-v2/`.

10 Bugs Encountered and Fixed

10.1 window.SKINAI vs window.SkinAI

Symptom: Image upload worked (progress bar appeared) but no result was ever shown. **Root cause:** `app.js` exports the API client as `window.SkinAI` (mixed case). Both `scan.html` and `index.html` referenced `window.SKINAI` (all caps). In JavaScript, property lookup is case-sensitive. `window.SKINAI` was `undefined`, so calling `undefined.runRealAnalysis()` threw a silent error.

Fix: Change `const S = window.SKINAI` to `const S = window.SkinAI` in both HTML files.

Lesson: When a UI interaction silently fails with no error shown to the user, always check the browser console. The console showed `TypeError: Cannot read properties of undefined (reading 'runRealAnalysis')` immediately.

10.2 Temperature scaling direction reversal

Covered in detail in Section 6.3. The short version: dividing a negative number by $T < 1$ makes it more negative (dampens the class). We had set rare class temperatures below 1 and dominant class temperatures above 1, which was exactly backwards.

Lesson: When implementing a mathematical formula, always sanity-check with a specific numerical example *before* deploying. We should have computed: "for a class with $p = 0.174$, what does $\log(0.174)/0.7$ equal?" $(= -2.5)$ and compared it to the un-scaled $\log(0.174) = -1.75$. The scaled value is *more negative* — this immediately reveals the bug.

10.3 False positives from top-3 high-risk check

After setting $T_{SCC} = T_{AK} = 8.0$, the calibrated probabilities for SCC and AK became very large on *all* images, not just true SCC/AK images. A true BKL image might get AK = 0.26 as rank-3. If we flag "high risk" whenever any of the top-3 predictions is in {MEL, BCC, SCC, AK} with confidence > 0.05, nearly every image would be flagged.

Fix: Revert to checking only the rank-1 prediction:

```
"high_risk": top[0].code in HIGH_RISK_CODES
```

This works correctly because the temperature scaling pushes genuine SCC/AK signal to rank-1 on true SCC/AK images. The argmax is a harder threshold but avoids systematic false positives.

Lesson: When boosting minority class scores by large factors, the boost applies to *all* images. A high SCC score does not necessarily mean the image is SCC when $T_{SCC} = 8$; it means the model had *any* SCC signal. The argmax criterion is more robust than a threshold on a temperature-inflated score.

11 Metrics and Honest Evaluation

11.1 Prevalence-weighted accuracy

For a triage tool deployed in a real clinic, the distribution of incoming cases matters. If 99% of cases are benign moles (NV), then correctly identifying NV is far more important than correctly identifying the rare SCC. The **prevalence-weighted accuracy** is:

$$\text{PWA} = \sum_c \pi_c \cdot \text{Recall}_c$$

where π_c is the real-world prevalence of class c. With approximate dermatology clinic prevalences (NV: 50%, BKL: 20%, MEL: 10%, BCC: 8%, AK: 5%, others: 1% each), the raw ensemble achieves approximately 75% PWA even though balanced accuracy is only 34%.

Important: PWA is not a substitute for balanced accuracy in a medical context. Missing a melanoma because it appeared in the 10% prevalence tail is not acceptable even if the aggregate score looks good. We report both metrics.

11.2 Full metric table (current production)

Class	Raw	Temp. scaling	LR head	Post-retrain (target)
MEL	40%	70%	100%	—
NV	95%	60%	88%	—
BCC	25%	50%	88%	—
BKL	35%	60%	88%	—
VASC	40%	80%	88%	—
AK	0%	30%	38%	60–70%
DF	0%	0%	38%	55–65%
SCC	0%	10%	12%	55–65%
Balanced	34%	41%	**67%**	**80+%**

Table 6: Recall at each calibration stage. Post-retrain targets are estimates based on ISIC ablation studies with LDAM.

12 Skills and Concepts for Future Projects

This section distils transferable lessons for use in similar ML engineering projects.

12.1 The 80% accuracy trap

Any time someone reports "our model achieves X% accuracy" on a classification task, ask:

1. What is the class distribution in the test set?
2. What does a naive baseline (always predict the majority class) achieve?
3. What is the balanced accuracy / macro-F1?

If accuracy $\gg$ balanced accuracy, the model is exploiting class imbalance. In medical contexts, *always* report balanced accuracy and per-class recall.

12.2 Diagnosing without labels

We did not have a labelled test set at the start of the project. We acquired one dynamically using the ISIC API. The general principle: **public APIs with labelled data are available for many domains** (medical imaging: ISIC; pathology: TCGA; radiology: NIH Chest X-ray). When evaluating a deployed model, always test on fresh data from an independent source, not the training distribution.

12.3 Post-hoc calibration hierarchy

1. **Temperature scaling** (single T): corrects overall overconfidence. Cannot address class imbalance.
2. **Per-class temperature scaling**: corrects per-class biases. Cannot learn cross-class patterns. Has ceiling at backbone signal strength.
3. **Logistic regression head on log-probs**: learns cross-class patterns. Strict generalisation of temperature scaling. Cannot recover backbone failures.
4. **Backbone fine-tuning with LDAM**: modifies the internal features. The only true fix for backbone failures.

Apply them in order: each is faster and cheaper than the next, and the limitations of each tell you whether to proceed to the next step.

12.4 Serverless GPU workflow

Modal is a Python-native serverless GPU provider. The pattern for ML workloads:

```
import modal

app = modal.App("my-ml-app")
volume = modal.Volume.from_name("model-weights",
   create_if_missing=True)

@app.function(gpu="T4", volumes={"/weights": volume})
def train_or_infer(args):
```

```
    # runs remotely on T4
    ...

@app.local_entrypoint()
def main(my_arg: str = "default"):
    # runs locally; my_arg passed via: modal run script.py --my
       -arg value
    result = train_or_infer.remote(my_arg)
```

Key patterns:

- Store large files (model weights, datasets) in **Volumes**, not container images.
- Use `@app.local_entrypoint()` with typed function parameters — Modal parses CLI args from the function signature. Do not use `argparse`.
- `scaledown_window` keeps the container warm after the last request, avoiding cold starts for interactive use.
- `modal volume put` uploads local files to a Volume for use in remote functions.

12.5 Ensemble design checklist

When building a model ensemble for a production ML system:

1. Use models of different *capacities* (not just different random seeds). Different capacity ⇒ different inductive biases ⇒ different errors.
2. Use **soft voting** (average probability vectors), not hard voting (majority vote). Soft voting preserves calibration information.
3. Add **TTA** for any task where the input can appear in multiple orientations.
4. Measure ensemble calibration (reliability diagrams) — ensembles can be well-calibrated or poorly calibrated depending on how the members are combined.

13 Codebase Reference

13.1 Re-running calibration after backbone retrain

```
# 1. Run backbone fine-tuning (takes 4-8h on T4, ~$3-5)
modal run scripts/fine_tune_modal.py

# 2. Update modal_app.py to point at finetuned weights
#    Change: ENSEMBLE_FILES = ("finetuned_efficientnet_b4.h5",
   ...)

# 3. Retrain the LR calibration head on the improved backbone
python scripts/train_calibrated_head.py --per-class 50 --skip
   20

# 4. Upload new head to Modal volume
modal volume put skinai-weights backend/calibration/head.pkl /
   calibration/head.pkl
```

File	Purpose
backend/inference.py	Core inference: TTA, ensemble averaging, calibration pipeline
backend/main.py	FastAPI endpoints; CORS; file upload handling
backend/modal_app.py	Modal deployment config: T4 GPU, volume mount, scikit-learn
backend/calibration/head.pkl	Trained LR calibration head (pickle, sklearn Pipeline)
scripts/train_calibrated_head.py	Retrains LR head from ISIC API; saves head.pkl
scripts/fine_tune_modal.py	LDAM fine-tuning of EfficientNet backbone on Modal T4
scripts/eval_confusion_matrix.py	Fresh ISIC evaluation; per-class recall
scripts/benchmark.py	Full benchmark suite; HTML report with charts
docs/benchmark_report.tex	Academic PDF report of model performance
web-v2/app.js	Frontend API client: window.SkinAI.runRealAnalysis()
web-v2/index.html	Landing page with live inference demo
web-v2/scan.html	Full screening interface
vercel.json	Vercel config: outputDirectory: web-v2, no build step

Table 7: Key files and their roles.

```
# 5. Redeploy
modal deploy backend/modal_app.py

# 6. Evaluate
python scripts/eval_confusion_matrix.py
```

14 Further Reading

- **EfficientNet**: Tan & Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML 2019. https://arxiv.org/abs/1905.11946
- **LDAM**: Cao et al., "Learning Imbalanced Datasets with Label-Distribution-Aware Margin Loss," NeurIPS 2019. https://arxiv.org/abs/1906.07413
- **Temperature scaling**: Guo et al., "On Calibration of Modern Neural Networks," ICML 2017. https://arxiv.org/abs/1706.04599
- **ISIC Archive**: International Skin Imaging Collaboration. https://www.isic-archive.com
- **HAM10000**: Tschandl et al., "The HAM10000 dataset," Sci. Data 2018. https://doi.org/10.1038/sdata.2018.161
- **EVA-02**: Fang et al., "EVA-02: A Visual Representation Powerhouse," 2023. https://arxiv.org/abs/2303.11331

15 Final Architecture Decision: SCC Removal (June 2026)

After exhausting every calibration and backbone approach, SCC recall remained at 0% across all tested architectures:

- EfficientNet B4/B5/B7 ensemble + temperature scaling: 0%
- EfficientNet + calibrated LR head: 0%
- EVA-02-Base + LR head: 0%
- Joint GBM (EfficientNet log-probs + EVA-02 PCA features): 0%
- ConvNeXt-Large fine-tuned on HAM10000: 0%
- ViT-Large fine-tuned on 12-class skin dataset: 0%

Root cause: The ISIC Archive API query used for SCC training data (`diagnosis_2 = "Malignant epidermal proliferations"`) is a parent category that returns a mixture of SCC and AK images with no way to further filter at the `diagnosis_3` level (that field returns zero results for "Squamous cell carcinoma"). The resulting training set has contradictory labels — the same visual patterns appear under both AK and SCC — making the classification boundary unlearnable.

Decision: SCC was removed from the active output. The inference pipeline now zeroes the SCC probability slot after ensemble scoring and renormalises. The probability mass flows to AK and BCC — the morphologically nearest high-risk classes — preserving the high-risk referral pathway for patients who actually have SCC.

Clinical note: Early SCC (Bowen's disease / SCC in situ) is morphologically indistinguishable from AK by dermoscopy alone. Both are pre-malignant keratotic lesions, both trigger the high-risk flag in SkinAI, and both warrant dermatologist referral. The removal therefore has minimal clinical impact on the screening workflow.